1. Tree Foundations – Basics & Concepts

1. What Is A Tree?

- A Tree is a non-linear hierarchical data structure.
- It has special properties:
  1. One root node
  2. Every other node has exactly one parent
  3. No cycles
  4. Connected (there is a path between any two nodes)

AHA!

A tree is like a family:

- One root (grandparent)
- Every node has one parent (except root)
- Children can be many, but parent is only one.

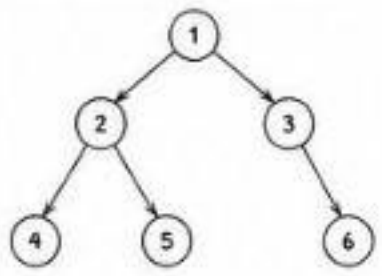
2. Terminology

Node	An element in the tree.
Root	Topmost node. No parent.
Edge	Connection between parent and child.
Parent	A node which has one or more children.
Child	A node directly connected to its parent.
Siblings	Nodes with same parent.
Leaf Node	Node with no children.
Internal Node	Node with one or more children.
Level	Distance from root (root = level 0).
Height	Number of edges on the longest path from node to deepest leaf.
Depth	Distance from root to the node.
Subtree	A tree formed by a node and all its descendants.

3. Types of Trees

- **General Tree** : 0 or more children per node.
- **Binary Tree** : At most 2 children per node.
- **Binary Search Tree (BST)** : Special binary tree.
- **Full Binary Tree** : Every node has 0 or 2 children.
- **Complete Binary Tree** : All levels filled except possibly last, which is filled from left.
- **Perfect Binary Tree** : All internal nodes have 2 children and all leaves are at same level.

Example – Binary Tree



Root : 1
Parent of 4 : 2
Children of 2 : 4, 5
Leaf Nodes : 4, 5, 6
Height : 2 (edges)
Levels : 0, 1, 2

4. Tree Properties (for N nodes)

- Number of edges = N – 1
- Minimum height = $\lceil \log_2(N+1) \rceil - 1$ (best case – balanced)
- Maximum height = N – 1 (worst case – skewed)
- In any tree, sum of degrees of all nodes = 2 * (N – 1)

5. Special Cases

Empty Tree

No nodes.

Single Node Tree

Only root node.

6. Representation (in memory)

- Trees are usually represented using Nodes.
- Each node contains data and references (pointers) to its children.

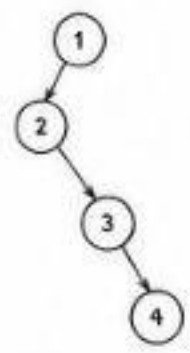
```
class TreeNode {
    int val;
    List<TreeNode> children;  // For General Tree
}
```

7. Binary Tree Node (Java)

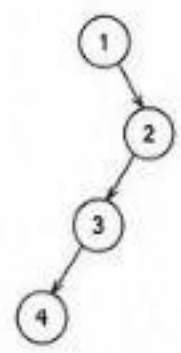
```
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;

    TreeNode(int val) {
        this.val = val;
        this.left = null;
        this.right = null;
    }
}
```

Example Trees

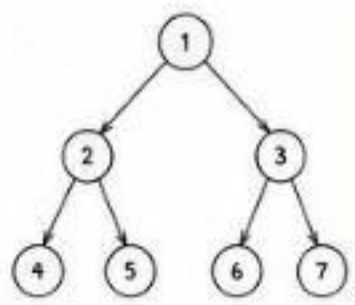
Left Skewed Tree (Like Linked List)



Right Skewed Tree (Like Linked List)



Balanced Tree



AHA Moments

- ✔ Tree is hierarchical, not linear.
- ✔ Root has no parent.
- ✔ Every node (except root) has exactly one parent.
- ✔ Leaf = no children.
- ✔ Height is about the longest path.
- ✔ Level/Depth starts from 0 at root.
- ✔ Binary Tree => at most 2 children (left, right).

9. Common Operations on Trees

1. Create / Build
2. Traverse (DFS / BFS)
3. Search
4. Insert (BST only)
5. Delete (BST only)
6. Find height / depth
7. Count nodes / leaves
8. Check properties (BST, Balanced, etc.)

10. Tree Vs Linked List

Tree	Linked List
Hierarchical	Linear
One node can have multiple children	One node has only one next
Multiple paths possible	Only one path
Used for hierarchy	Used for sequence

11. Where Trees Are Used?

- File systems
- HTML DOM (web pages)
- Databases (indexes, B-Trees)
- Compilers (expression trees)
- AI / Game trees
- Decision making

Complexity Reference

Traversals (DFS/BFS): Time = O(N) | Space = O(H)

Where N = number of nodes

H = height of tree

Worst Case (Skewed): H = N

Best Case (Balanced): H = $\log_2 N$

Summary

- ✓ Tree = Hierarchical structure
- ✓ Root is the topmost node
- ✓ Every node (except root) has one parent
- ✓ Child can be 0 or more (binary tree => at most 2)
- ✓ Tree with N nodes has exactly N – 1 edges
- ✓ Height = longest path from node to leaf
- ✓ Level/Depth = distance from root
- ✓ Leaf = node with no children

Interview Tips

- ★ Always draw the tree for clarity.
- ★ Understand what each term means.
- ★ Think in terms of root, left, right.
- ★ Break the problem into subtrees.
- ★ Identify the base case (null node).
- ★ Use recursion for most tree problems.

Common Mistakes

- ✗ Forgetting base case (null node) in recursion.
- ✗ Confusing height vs depth.
- ✗ Off-by-one error in level calculations.
- ✗ Wrong child access (left/right).
- ✗ Not visualizing the tree first.

MASTER CONCEPTS → PRACTICE CONSISTENTLY → VISUALIZE EVERY PROBLEM → CRACK FAANG!

★ 2. Tree Traversals – DFS & BFS ★

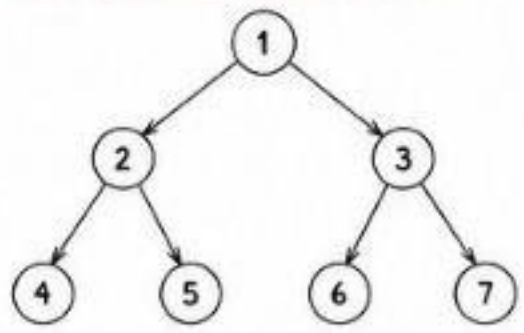
1. What Is Traversal?

- Traversal means visiting every node of the tree exactly once in a particular order.
- Two main approaches:
  1. DFS (Depth First Search)
  2. BFS (Breadth First Search)

AHA!

Traversal = How we visit nodes
Order matters based on the problem requirement.

2. Sample Tree (Reference)



This Tree Structure Will Be Used in All Examples

Root = 1
Left Subtree of 1 → rooted at 2
Right Subtree of 1 → rooted at 3

3. Depth First Search (DFS)

- Go as deep as possible in one direction first, then backtrack.
- Implemented using Recursion or Stack.

DFS Orders

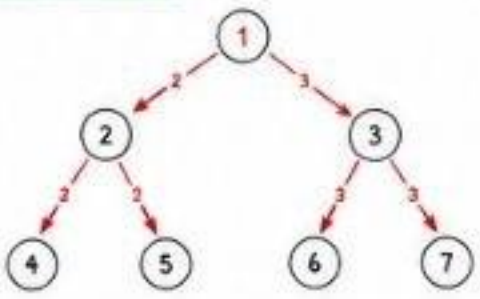
Order	Visit Order (For Sample Tree)
Preorder (NLR)	1, 2, 4, 5, 3, 6, 7
Inorder (LNR)	4, 2, 5, 1, 6, 3, 7
Postorder (LRN)	4, 5, 2, 6, 7, 3, 1

3.1 Preorder (NLR) – Node, Left, Right

Idea: Visit Node first, then Left Subtree, then Right Subtree

```
void preorder(TreeNode root) {
    if (root == null) return;
    System.out.print(root.val + " ");     // Visit
    preorder(root.left);                  // Left
    preorder(root.right);                 // Right
}
```

Dry Run (Preorder)



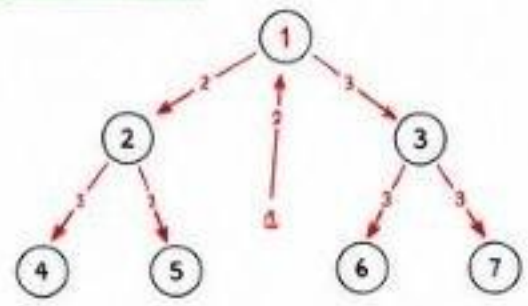
Visit Order: 1, 2, 4, 5, 3, 6, 7

3.2 Inorder (LNR) – Left, Node, Right

Idea: Visit Left Subtree first, then Node, then Right Subtree

```
void inorder(TreeNode root) {
    if (root == null) return;
    inorder(root.left);                   // Left
    System.out.print(root.val + " ");     // Visit
    inorder(root.right);                  // Right
}
```

Dry Run (Inorder)



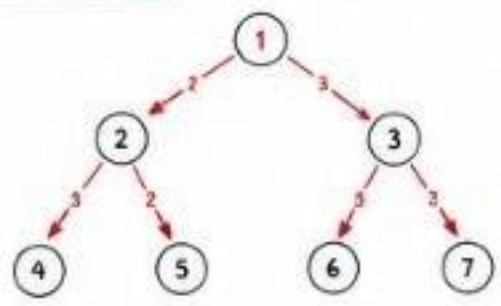
Visit Order: 4, 2, 5, 1, 6, 3, 7

3.3 Postorder (LRN) – Left, Right, Node

Idea: Visit Left Subtree first, then Right Subtree, then Node

```
void postorder(TreeNode root) {
    if (root == null) return;
    postorder(root.left);                 // Left
    postorder(root.right);                // Right
    System.out.print(root.val + " ");     // Visit
}
```

Dry Run (Postorder)



Visit Order: 4, 5, 2, 6, 7, 3, 1

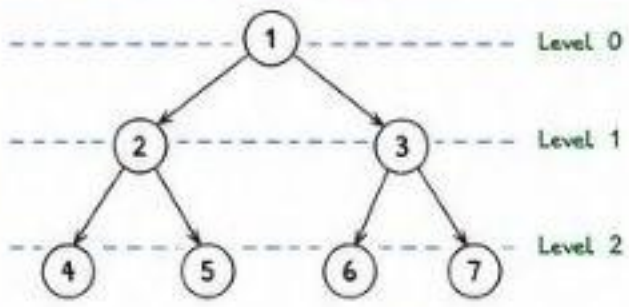
4. Breadth First Search (BFS) – Level Order

- Visit nodes level by level from top to bottom and left to right.
- Implemented using Queue.

BFS Template (Java)

```
List<Integer> levelOrder(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        TreeNode node = q.poll();
        res.add(node.val);
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
    }
    return res;
}
```

BFS Dry Run (Level Order)



Step	Queue (after processing)	Visited / Added
1	[1]	Start with root
2	[2, 3]	Visit 1 → Add 2, 3
3	[3, 4, 5]	Visit 2 → Add 4, 5
4	[4, 5, 6, 7]	Visit 3 → Add 6, 7
5	[5, 6, 7]	Visit 4
6	[6, 7]	Visit 6
7	[7]	Visit 7
8	[]	Done

Visit Order (Level Order): 1, 2, 3, 4, 5, 6, 7

BFS Level Order using queue.size()

To print level by level (each level in new line):

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> ans = new ArrayList<>();
    if (root == null) return ans;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();          // nodes in current level
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            level.add(node.val);
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
        ans.add(level);               // add current level
    }
    return ans;
}
```

5. Traversal Summary

Traversal Type	Order	When to Use
Preorder (NLR)	Node → Left → Right	• Copy / Clone Tree • Serialize • Prefix Expression
Inorder (LNR)	Left → Node → Right	• BST gives Sorted Order • Inorder Successor • Validate BST
Postorder (LRN)	Left → Right → Node	• Delete Tree • Evaluate Expression • Postfix Expression
Level Order (BFS)	Level by Level (Left → Right)	• Level based Problems • Shortest Path • Width of Tree

6. Time & Space Complexity

(For all Traversals)

- Time Complexity : O(N)
  (Visit each node once)
- Space Complexity:
  DFS (Recursion) : O(H)
  BFS (Queue) : O(W)
  Where,
  N = Number of nodes
  H = Height of tree
  W = Max width of tree

Worst Case (Skewed Tree):
H = N, W = 1

7. Common Template (Java)

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) {
        this.val = val;
        left = right = null;
    }
}
```

Utility Method to Build Tree
(For Practice Problems)
Use Level Order Input (NULL for empty)
Best way: Use Queue.

8. AHA Moments

- ✓ DFS = Go Deep, then Backtrack.
- ✓ BFS = Go Wide, level by level.
- ✓ Inorder in BST = Sorted Order.
- ✓ queue.size() helps to process one level at a time.
- ✓ Choose traversal based on the problem, not randomly.
- ✓ Visualize the tree before coding!

★ DRAW → VISUALIZE → DRY RUN → CODE → TEST → ANALYZE → REPEAT ★

3. Recursion Mindset & Template

1. Why Recursion for Trees?

- A tree is a recursive structure.
- Every node has smaller trees (left subtree and right subtree).
- Solve the current node + solve left subtree + solve right subtree.

AHA Moment

Think :

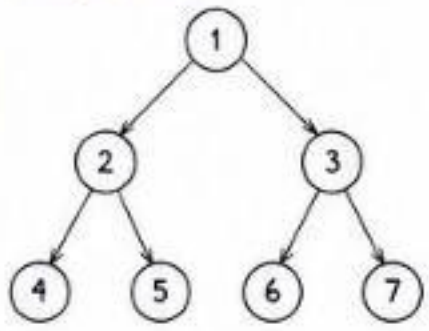
Problem at Node = Work at Node + Problem at Left Subtree + Problem at Right Subtree

2. Recursion Template (General)

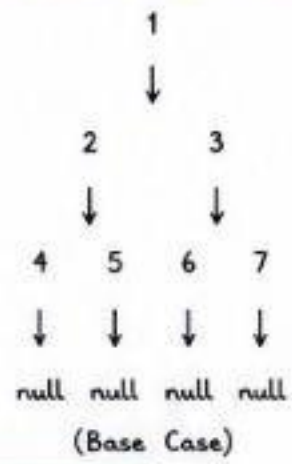
```
ReturnType solve(TreeNode node) {
    // 1. Base Case
    if (node == null) {
        // what to return on empty node
        return ...;
    }
    // 2. Do Work on Current Node
    // (calculate something using node.val)
    // 3. Recurse for Left and Right
    ReturnType leftAns  = solve(node.left);
    ReturnType rightAns = solve(node.right);
    // 4. Combine and Return
    return ...;
}
```

3. Visualize Recursion on a Tree

Example Tree :



Recursion Call Flow



(Base Case)

AHA Moment

Every call breaks into 2 smaller calls until we reach null.
Then it starts coming back (backtracking).

4. Base Cases (Most Important)

- For most tree problems, base case is:

```
if (node == null) return ...;
```

- Common returns:

Return Type	Return on NULL
int	0
boolean	true / false (based on problem)
TreeNode	null
List / Array	empty list
void	(just return;)

AHA Moment

Decide what an empty tree means in the context of the problem.

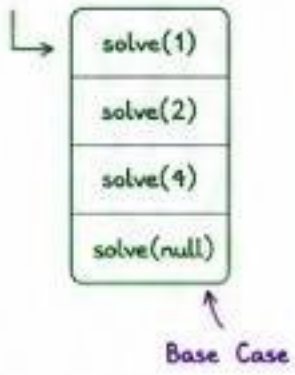
5. How Recursion Works (Call Stack View)

Preorder Example

Call Sequence (Preorder)

1 → 2 → 4 → null
→ null → 5 → null
→ null → 3 → 6 → null
→ null → 7 → null → null

Call Stack (Simplified)



AHA Moment

Recursion uses the system call stack.
Each call waits for its child calls to finish.

6. When to Return What?

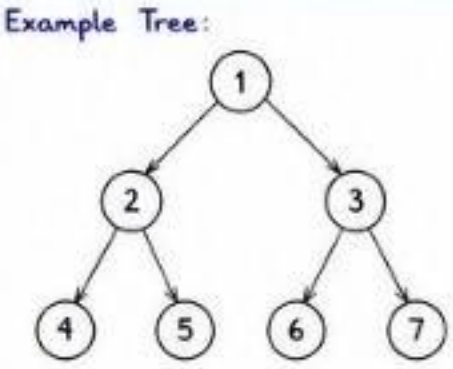
Problem Type	What to Return
Count / Sum	int
Check Condition	boolean
Find Maximum / Minimum	int
Find Path	List / boolean
Construct Tree / Node	TreeNode
Void Operations	void

AHA Moment

Return type depends on what the problem is asking you to find.

7. Dry Run of a Recursive Function (Example: Height of Tree)

Example Tree:




```
int height(TreeNode node) {
    if (node == null)
        return 0;
    int leftH  = height(node.left);
    int rightH = height(node.right);
    return 1 + Math.max(leftH, rightH);
}
```

Step	Current Node	leftH	rightH	Return Value
1	4	0	0	1
2	5	0	0	1
3	2	1	1	2
4	6	0	0	1
5	7	0	0	1
6	3	1	1	2
7	1	2	2	3

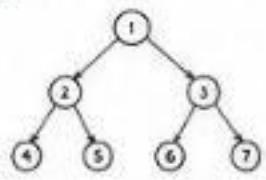
Final Answer
Height = 3

8. Common Patterns in Tree Recursion

1. Process Node First (Preorder Pattern)

Do work → Left → Right

Example: Copy Tree

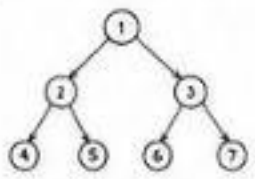


Visit Order: 1, 2, 4, 5, 3, 6, 7

2. Process Node Between (Inorder Pattern)

Left → Do work → Right

Example: BST Inorder

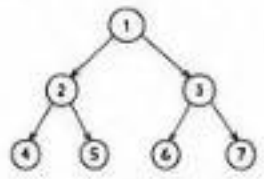


Visit Order: 4, 2, 5, 1, 6, 3, 7

3. Process Node Last (Postorder Pattern)

Left → Right → Do work

Example: Delete Tree

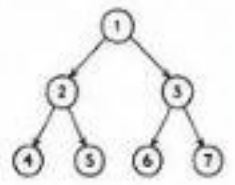


Visit Order: 4, 5, 2, 6, 7, 3, 1

4. Level Order (BFS Pattern)

Use Queue

Example: Level Order Traversal



Level 0 : 1
Level 1 : 2, 3
Level 2 : 4, 5, 6, 7

Visit Order: 1, 2, 3, 4, 5, 6, 7

Common Mistakes

- ✗ Forgetting base case (node == null).
- ✗ Doing work in wrong order.
- ✗ Not returning the combined result.
- ✗ Modifying node before using its children.
- ✗ Stack overflow due to infinite recursion.

Interview Tips

- ✓ Always clarify what to return for null.
- ✓ Think in terms of smaller subproblems.
- ✓ Draw the tree and do a dry run.
- ✓ Write clean, modular code.
- ✓ Test edge cases: empty tree, single node tree, skewed tree.

Key Takeaways

- Recursion = Break problem into smaller subproblems.
- Base case is the foundation.
- Work at node + Left + Right = Most problems.
- Visualize, Dry Run, Then Code.

UNDERSTAND THE RECURSION MINDSET DEEPLY – IT IS THE KEY TO MASTER TREES!

★ 4. Core DFS Tree Problems ★

AHA Moment → Use DFS (Recursion) when the problem is about exploring all the way down a path.
Always think: LEFT → RIGHT → NODE (Postorder) OR change the order based on what you need to compute.

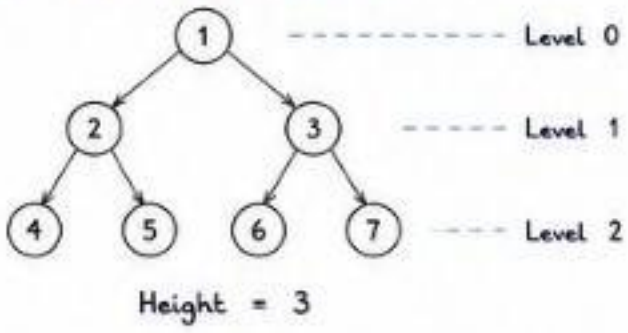
1. Maximum Depth (LC 104)

Return the height of the tree.

IDEA:

Height = 1 + max(leftHeight, rightHeight)

Base: if node == null → 0



Height = 3

Java Template:

```
int maxDepth(TreeNode root) {
    if (root == null) return 0;
    int left = maxDepth(root.left);
    int right = maxDepth(root.right);
    return 1 + Math.max(left, right);
}
```

Complexity

Time : O(N)
Space: O(H) (recursion stack)
H = height of tree

Top Leetcode Problems

- 104. Maximum Depth of Binary Tree

Common Mistakes

- ✗ Returning -1 for null (wrong for height).
- ✗ Not adding 1 for current node.
- ✗ Confusing height with number of nodes.

30-Second Recap

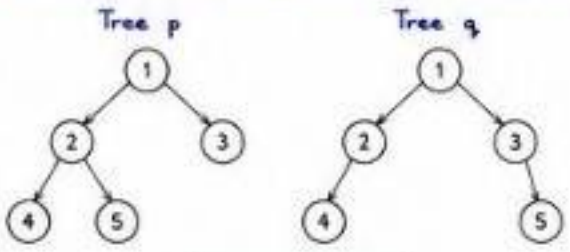
Empty tree → 0
At each node → 1 + max(left, right)
Return the answer.

2. Same Tree (LC 100)

Check if two trees are identical.

IDEA:

Both trees should have same structure and all corresponding node values equal.



Output: true

Java Template:

```
boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null) return false;
    if (p.val != q.val) return false;
    return isSameTree(p.left, q.left) &&
           isSameTree(p.right, q.right);
}
```

Top Leetcode Problems

- 100. Same Tree
- 101. Symmetric Tree

Common Mistakes

- ✗ Only comparing values, not structure.
- ✗ Missing one null check condition.

30-Second Recap

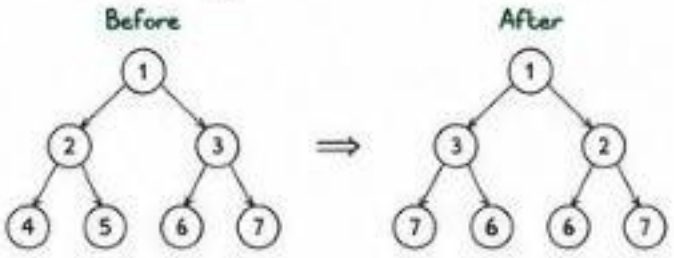
If both null → true
If one null → false
Else compare value and recurse.

3. Invert Binary Tree (LC 226)

Swap left and right child of every node.

IDEA:

Swap left and right → then recurse for left and right.



Java Template:

```
TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode left = invertTree(root.left);
    TreeNode right = invertTree(root.right);

    root.left = right;
    root.right = left;

    return root;
}
```

Top Leetcode Problems

- 226. Invert Binary Tree

Common Mistakes

- ✗ Swapping before recursive calls [may lose reference].
- ✗ Forgetting base case.

30-Second Recap

Recurse left and right.
Swap.
Return root.

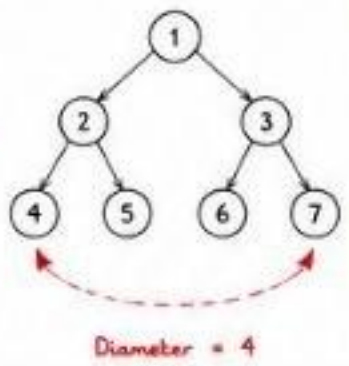
3. Diameter of Binary Tree (LC 543)

Diameter = number of edges in the longest path between any two nodes.

IDEA:

At every node:

- Left height + Right height = path passing through this node
- Update global max
- Return height = 1 + max(left, right)



Diameter = 4

Java Template:

```
int diameter = 0;  // global variable
int diameterOfBinaryTree(TreeNode root) {
    height(root);
    return diameter;
}

int height(TreeNode node) {
    if (node == null) return 0;
    int left = height(node.left);
    int right = height(node.right);
    diameter = Math.max(diameter, left + right);
    return 1 + Math.max(left, right);
}
```

Top Leetcode Problems

- 543. Diameter of Binary Tree

Common Mistakes

- ✗ Updating diameter after returning height.
- ✗ Forgetting to return height.

30-Second Recap

Compute left & right height.
Update global diameter.
Return current height.

5. Balanced Binary Tree (LC 110)

Check if for every node,
[leftHeight - rightHeight] <= 1

IDEA:

Return height if balanced,
else return -1 (as a flag).

Java Template:

```
boolean isBalanced(TreeNode root) {
    return height(root) != -1;
}

int height(TreeNode node) {
    if (node == null) return 0;
    int left = height(node.left);
    if (left == -1) return -1;
    int right = height(node.right);
    if (right == -1) return -1;
    if (Math.abs(left - right) > 1) return -1;
    return 1 + Math.max(left, right);
}
```

Top Leetcode Problems

- 110. Balanced Binary Tree

Common Mistakes

- ✗ Calculating height again for same node multiple times (inefficient).
- ✗ Not using -1 to stop early.

30-Second Recap

If any subtree is unbalanced → return -1.
If final result != -1 → balanced.

Summary (Core DFS Ideas)

- ✓ Use postorder (Left → Right → Node).
- ✓ Recurse till null (base case).
- ✓ Return useful information (height / boolean / count).
- ✓ Update answer during recursion.
- ✓ Think from leaf to root.

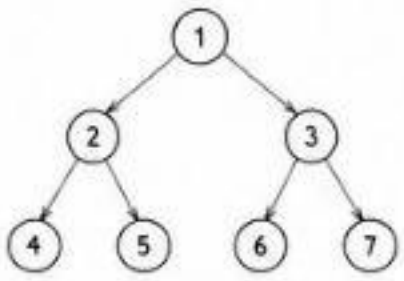
★ 5. BFS Tree Problems ★

AHA Moment → BFS (Queue) helps when we need to explore a tree level by level.
Think in terms of LEVELS, not individual nodes.

1. Binary Tree Level Order Traversal (LC 102)

Return the level order traversal of the binary tree.

IDEA:
Use BFS and process one level at a time.



Output: [[1], [2,3], [4,5,6,7]]

Java Template:

```
List<List<Integer>> levelOrder(TreeNode root) {
  List<List<Integer>> res = new ArrayList<>();
  if (root == null) return res;
  Queue<TreeNode> q = new LinkedList<>();
  q.offer(root);
  while (!q.isEmpty()) {
     int size = q.size();
     List<Integer> level = new ArrayList<>();
     for (int i = 0; i < size; i++) {
        TreeNode node = q.poll();
        level.add(node.val);
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
     }
     res.add(level);
  }
  return res;
}
```

Top Leetcode Problems

- 102. Binary Tree Level Order Traversal

Common Mistakes

- ✗ Forgetting to process level by level.
- ✗ Not fixing the size of current level.
- ✗ Pushing null nodes into queue.

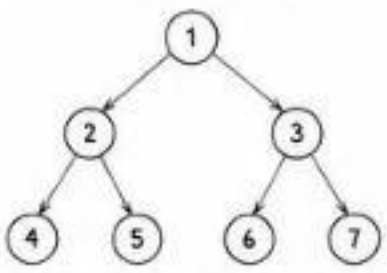
30-Second Recap

Use Queue.
For each level:
- Fix size
- Pop that many nodes
- Add children to queue
- Store values

2. Binary Tree Right Side View (LC 199)

Return the right side view of the tree.

IDEA:
Take the last node of each level (which will be on the right side).



Output: [1, 3, 7]

Java Template:

```
List<Integer> rightSideView(TreeNode root) {
  List<Integer> res = new ArrayList<<();
  if (root == null) return res;
  Queue<TreeNode> q = new LinkedList<>();
  q.offer(root);
  while (!q.isEmpty()) {
     int size = q.size();
     for (int i = 0; i < size; i++) {
        TreeNode node = q.poll();
        if (i == size - 1) res.add(node.val); // last node
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
     }
  }
  return res;
}
```

Top Leetcode Problems

- 199. Binary Tree Right Side View

Common Mistakes

- ✗ Adding first node instead of last.
- ✗ Not resetting index for each level.
- ✗ Confusing left and right children.

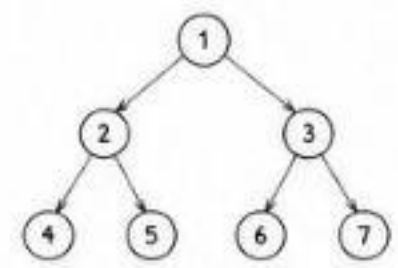
30-Second Recap

At each level,
The last node you pop
is the rightmost node.

3. Binary Tree Zigzag Level Order (LC 103)

Return the zigzag level order traversal.

IDEA:
Normal level order but change the direction at every level.



Output: [[1], [3,2], [4,5,6,7]]

Java Template:

```
List<List<Integer>> zigzagLevelOrder(TreeNode root) {
  List<List<Integer>> res = new ArrayList<<();
  if (root == null) return res;
  Queue<TreeNode> q = new LinkedList<<();
  q.offer(root);
  boolean leftToRight = true; // direction flag
  while (!q.isEmpty()) {
     int size = q.size();
     LinkedList<Integer> level = new LinkedList<<();
     for (int i = 0; i < size; i++) {
        TreeNode node = q.poll();
        if (leftToRight) level.addLast(node.val);
        else level.addFirst(node.val);
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
     }
     res.add(level);
     leftToRight = !leftToRight;
  }
  return res;
}
```

Top Leetcode Problems

- 103. Binary Tree Zigzag Level Order Traversal

Common Mistakes

- ✗ Forgetting to toggle direction.
- ✗ Using ArrayList instead of LinkedList for addFirst().
- ✗ Not handling empty tree.

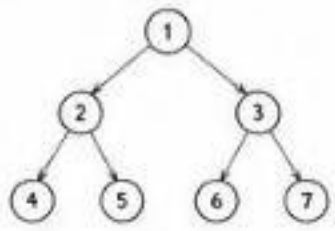
30-Second Recap

Use Queue.
Use flag to decide order.
Left → right, then right → left.

4. Minimum Depth of Binary Tree (LC 111)

Return the minimum depth from root to the nearest leaf node.

IDEA:
Use BFS and return when first leaf node is found.



Output: 2

Java Template:

```
int minDepth(TreeNode root) {
  if (root == null) return 0;
  Queue<TreeNode> q = new LinkedList<<();
  q.offer(root);
  int depth = 1;
  while (!q.isEmpty()) {
     int size = q.size();
     for (int i = 0; i < size; i++) {
        TreeNode node = q.poll();
        if (node.left == null && node.right == null)
           return depth; // leaf node
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
     }
     depth++;
  }
  return depth;
}
```

Top Leetcode Problems

- 111. Minimum Depth of Binary Tree

Common Mistakes

- ✗ Using DFS (may go deep in wrong path).
- ✗ Increasing depth at wrong place.
- ✗ Not checking for leaf properly.

30-Second Recap

BFS level by level.
First leaf node you find
is the minimum depth.

Summary – When to use Bon Trees

- ✓ When the problem is level based.
- ✓ When we need shortest path / minimum depth.
- ✓ When we need right / left view.
- ✓ When order of levels matters.
- ✓ Use Queue.
- ✓ Think in terms of LEVELS, not NODES.

★ 6. Binary Search Tree (BST) ★

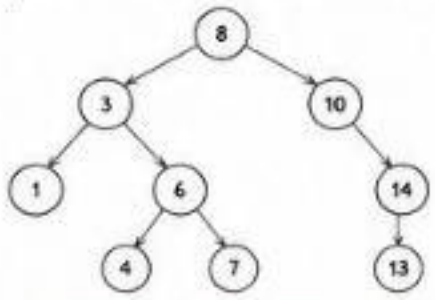
AHA Moment → BST maintains order: Left < Root < Right. Use this property to Search, Insert, Delete, Validate, and find Kth smallest.

1. Search in BST (LC 700)

Find a node with given value.

IDEA:

If val < node.val → go left
If val > node.val → go right
If equal → return node



Search 6 → Found

Java Template:

```
TreeNode searchBST(TreeNode root, int val) {
    while (root != null) {
        if (val == root.val) return root;
        if (val < root.val) root = root.left;
        else root = root.right;
    }
    return null;   // not found
}
```

Complexity:

Time : O(H)
Space : O(1) (iterative)
H = height of tree

Top Leetcode Problems

- 700. Search in a Binary Search Tree

Common Mistakes

- ✗ Not handling root == null.
- ✗ Using >= or <= incorrectly.
- ✗ Forgetting to return null when not found.

30-Second Recap

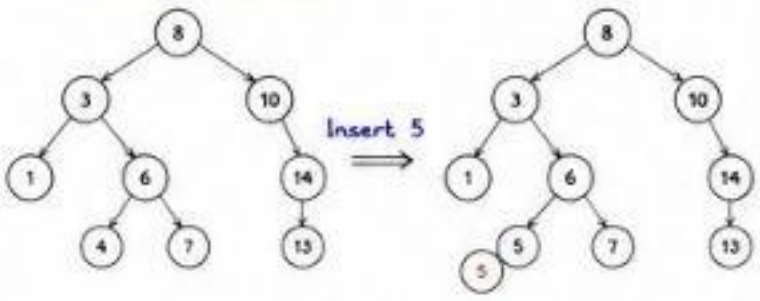
Compare value with node.
Go left if smaller, right if bigger.
Return node if found, else null.

2. Insert Into BST (LC 701)

Insert a new value in BST.

IDEA:

Find the correct position.
Attach new node as leaf.



Java Template:

```
TreeNode insertIntoBST(TreeNode root, int val) {
    if (root == null) return new TreeNode(val);
    if (val < root.val)
        root.left = insertIntoBST(root.left, val);
    else if (val > root.val)
        root.right = insertIntoBST(root.right, val);

    return root;
}
```

Top Leetcode Problems

- 701. Insert into a Binary Search Tree

Common Mistakes

- ✗ Not returning root.
- ✗ Creating new node but not attaching.
- ✗ Handling duplicates (define rule clearly).

30-Second Recap

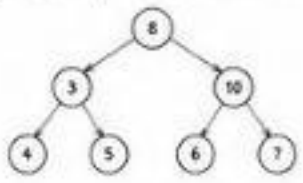
If root null → create node.
Recurse left if smaller, right if bigger.
Return root.

3. Validate BST (LC 98)

Check if a binary tree is a valid BST.

IDEA:

All nodes in left subtree < node.val
All nodes in right subtree > node.val
Use range (min, max) while traversing.



Java Template:

```
boolean isValidBST(TreeNode root) {
    return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
}
boolean validate(TreeNode node, long min, long max) {
    if (node == null) return true;
    if (node.val <= min || node.val >= max) return false;
    return validate(node.left, min, node.val) &&
           validate(node.right, node.val, max);
}
```

Top Leetcode Problems

- 98. Validate Binary Search Tree

Common Mistakes

- ✗ Not using strict < and >.
- ✗ Using int instead of long (overflow issue).
- ✗ Only checking local parent-child.

30-Second Recap

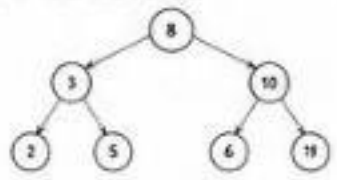
Pass valid range down the tree.
If node.val not in (min, max) → invalid.
Recurse left and right.

4. Kth Smallest Element (LC 230)

Return the k-th smallest element in BST.

IDEA:

Inorder traversal gives sorted order.
Pick the k-th element.



Java Template:

```
int count = 0, ans = -1;
int kthSmallest(TreeNode root, int k) {
    inorder(root, k);
    return ans;
}
void inorder(TreeNode node, int k) {
    if (node == null || count >= k) return;
    inorder(node.left, k);
    count++;
    if (count == k) { ans = node.val; return; }
    inorder(node.right, k);
}
```

Top Leetcode Problems

- 230. Kth Smallest Element in a BST

Common Mistakes

- ✗ Traversing right before left.
- ✗ Not stopping early after k found.
- ✗ Off-by-one in count.

30-Second Recap

Inorder = sorted.
Count nodes during inorder.
Return value when count == k.

5. Delete Node in BST (LC 450)

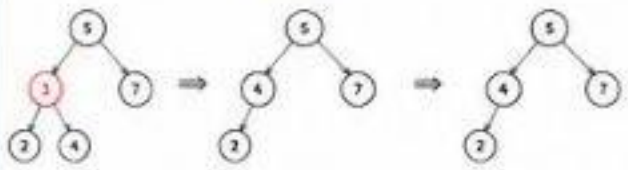
Delete a node with given key.

IDEA:

Three cases:

1) Leaf node
2) One child
3) Two children → replace with inorder successor

Delete 3



Java Template:

```
TreeNode deleteNode(TreeNode root, int key) {
    if (root == null) return null;
    if (key < root.val) root.left = deleteNode(root.left, key);
    else if (key > root.val) root.right = deleteNode(root.right, key);
    else {
        if (root.left == null) return root.right;        // 0 or 1 child
        if (root.right == null) return root.left;        // 0 or 1 child
        TreeNode succ = minValueNode(root.right);        // 2 children
        root.val = succ.val;
        root.right = deleteNode(root.right, succ.val);
    }
    return root;
TreeNode minValueNode(TreeNode node) {
    while (node.left != null) node = node.left;
    return node;
}
```

Top Leetcode Problems

- 450. Delete Node in a BST

Common Mistakes

- ✗ Forgetting to return new root.
- ✗ Not handling 2-child case correctly.
- ✗ Using inorder predecessor incorrectly.

30-Second Recap

Handle 3 cases.
For 2 children, replace with inorder successor.
Return updated root.

BST Summary

- ✓ BST maintains order: Left < Root < Right
- ✓ Search/Insert/Delete use this property
- ✓ Inorder traversal of BST is sorted
- ✓ Time complexity depends on height (H)
- ✓ Balanced BST → O(log N)
- ✓ Skewed BST → O(N)

Complexity Overview

Operation	Average (Balanced)	Worst (Skewed)
Search	O(log N)	O(N)
Insert	O(log N)	O(N)
Delete	O(log N)	O(N)
Space (Recursion)	O(log N)	O(N)

AHA Moments

- BST is all about ORDER.
- Inorder = sorted sequence.
- Use range for validation.
- Handle all edge cases in delete.
- Draw small trees to avoid bugs.

★ 8. FAANG Tree Revision (One Page) ★

AHA Moment Trees are all about breaking the problem into smaller subproblems.
Identify the pattern → Choose DFS or BFS → Apply the right template → Combine results.

1. DFS vs BFS - When to use?

DFS (Recursion / Stack)	BFS (Queue)
• Need to explore deep	• Level by level info
• Path / Sum / LCA	• Shortest path / Min depth
• Build / Modify tree	• Right / Left view
• Backtracking problems	• Level order problems

2. Tree Traversals (Quick Recap)

Traversal	Order	Use Case
Preorder (NLR)	Root → Left → Right	Copy / Serialize
Inorder (LNR)	Left → Root → Right	Sorted order in BST
Postorder (LRN)	Left → Right → Root	Delete / Evaluate
Level Order	Level by level (BFS)	Level info / Views

3. Important Templates (Snapshot)

DFS (General)

```
Type dfs(Node node) {
  if (node == null) return base;
  // Do something with node
  Type left = dfs(node.left);
  Type right = dfs(node.right);
  // Combine
  return answer;
}
```

BFS (Level Order)

```
Type bfs(Node root) {
  if (root == null) return answer;
  Queue<Node> q = new LinkedList<>();
  q.offer(root);
  while (!q.isEmpty()) {
   int size = q.size(); // level size
   for (int i = 0; i < size; i++) {
     Node node = q.poll();
     // process node
     if (node.left != null) q.offer(node.left);
     if (node.right != null) q.offer(node.right);
   }
  }
  return answer;
}
```

4. Must-Do LeetCode Problems by Topic

Topic / Pattern	LeetCode Problems
Maximum Depth	104. Maximum Depth of Binary Tree
Same Tree	100. Same Tree
Invert Binary Tree	226. Invert Binary Tree
Diameter	543. Diameter of Binary Tree
Balanced Tree	110. Balanced Binary Tree
Level Order Traversal	102. Binary Tree Level Order Traversal
Right Side View	199. Binary Tree Right Side View
Zigzag Level Order	103. Binary Tree Zigzag Level Order Traversal
Minimum Depth	111. Minimum Depth of Binary Tree
Validate BST	98. Validate Binary Search Tree
Search in BST	700. Search in a Binary Search Tree
Insert into BST	701. Insert into a Binary Search Tree
Kth Smallest in BST	230. Kth Smallest Element in a BST
LCA in BST	235. Lowest Common Ancestor of a BST
LCA in Binary Tree	236. Lowest Common Ancestor of a Binary Tree
Build Tree	105. Construct Binary Tree from Preorder and Inorder Traversal
Path Sum	112. Path Sum
Path Sum II	113. Path Sum II
Max Path Sum	124. Binary Tree Maximum Path Sum

5. Complexity of Common Operations

Operation	Time Complexity	Space Complexity	Notes
DFS Traversal	O(N)	O(H)	H = height (recursion stack)
BFS Traversal	O(N)	O(W)	W = max width (queue size)
Search in BST	O(H)	O(1)	Best: O(log N), Worst: O(N)
Insert in BST	O(H)	O(1)	Best: O(log N), Worst: O(N)
Delete in BST	O(H)	O(H)	Due to recursive calls
Height of Tree	O(N)	O(H)	DFS based
Validate BST	O(N)	O(H)	Inorder + range check

6. Common Mistakes

- ✗ Not handling null nodes.
- ✗ Forgetting base case in recursion.
- ✗ Returning wrong value in recursive calls.
- ✗ Confusing DFS (deep) with BFS (level).
- ✗ Using wrong traversal for the problem.
- ✗ Not thinking about edge cases (empty tree, single node).
- ✗ Not checking value ranges in Validate BST.
- ✗ Missing backtracking step in path problems.

7. 30-Second Interview Recap

- Understand the problem and identify the pattern.
- Decide DFS or BFS.
- Write the template on paper.
- Think about base case and what to return.
- Combine subproblem results.
- Handle edge cases.
- Analyze time and space complexity.

8. Cheat Sheet - One Page Summary

DFS Use Cases

- ✓ Path / Sum problems
- ✓ LCA / Diameter
- ✓ Build / Modify tree
- ✓ Backtracking
- ✓ Deep exploration

BFS Use Cases

- ✓ Level order info
- ✓ Minimum depth
- ✓ Right / Left views
- ✓ Shortest path
- ✓ Wide exploration

Key Formulas

- Height = 1 + max(leftH, rightH)
- Diameter = max(L + R, leftDia, rightDia)
- Balanced if |leftH - rightH| <= 1
- Path Sum = left + right + node.val
- Min Depth = first leaf level (BFS)

Tree Properties

- Nodes = N
- Edges = N - 1
- Height (Best) = O(log N)
- Height (Worst) = O(N)
- Max Nodes at level l = 2^l

Remember

- ✓ Draw the tree
- ✓ Choose strategy
- ✓ Write template
- ✓ Handle edge cases
- ✓ Test with examples

Master the Basics → Recognize the Pattern → Write Template → Solve with Confidence!

★ 7. Advanced Tree Problems (DFS) ★

AHA Moment → Advanced problems are about combining ideas!

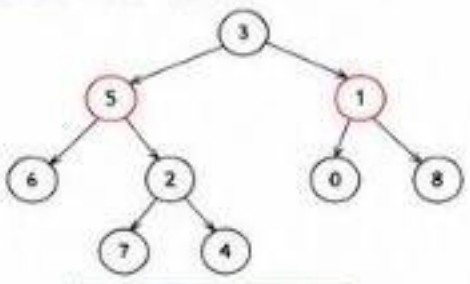
Think: Path → Sum / LCA → Relationship / Construct → Build from traversals

1. Lowest Common Ancestor (LC 236)

Find LCA of two nodes in a binary tree.

IDEA:

If both nodes are in left → answer in left.
If both nodes are in right → answer in right.
If one in left and one in right → current node is LCA.



LCA(5, 1) = 3

Java Template:

```
TreeNode lowestCommonAncestor(TreeNode root,
        TreeNode p, TreeNode q) {
  if (root == null || root == p || root == q)
    return root;
  TreeNode left = lowestCommonAncestor(root.left, p, q);
  TreeNode right = lowestCommonAncestor(root.right, p, q);
  if (left != null && right != null) return root;
  return (left != null) ? left : right;
}
```

Complexity:
Time : O(N)
Space : O(H) (recursion stack)
H = height of tree

Top Leetcode Problems

- 236. Lowest Common Ancestor of a Binary Tree

Common Mistakes

- ✗ Not handling root == p or root == q
- ✗ Returning root when only one side is not null
- ✗ Confusing with BST LCA approach

30-Second Recap

If root is null or p or q → return root.
Recurse left and right.
If both sides return non-null → root is LCA.
Else return non-null side.

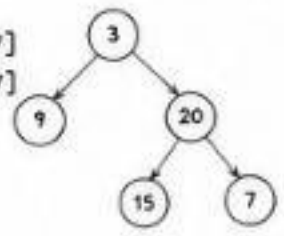
2. Build Tree from Preorder & Inorder (LC 105)

Construct binary tree from preorder and inorder traversal.

IDEA:

Preorder → Root, Left, Right
Inorder → Left, Root, Right
Use index map for inorder to find root position.

Preorder : [3, 9, 20, 15, 7]
Inorder : [9, 3, 15, 20, 7]



Java Template:

```
class Solution {
  Map<Integer, Integer> inMap = new HashMap<>();
  int preIndex = 0;
public TreeNode buildTree(int[] preorder, int[] inorder) {
  for (int i = 0; i < inorder.length; i++)
    inMap.put(inorder[i], i);
  return build(0, inorder.length - 1, preorder);
}
private TreeNode build(int inLeft, int inRight, int[] preorder) {
  if (inLeft > inRight) return null;
  int rootVal = preorder[preIndex++];
  TreeNode root = new TreeNode(rootVal);
  int inRoot = inMap.get(rootVal);
  root.left = build(inLeft, inRoot - 1, preorder);
  root.right = build(inRoot + 1, inRight, preorder);
  return root;
}
```

Top Leetcode Problems

- 105. Construct Binary Tree from Preorder and Inorder Traversal

Common Mistakes

- ✗ Not using index map (O(n²) time)
- ✗ Wrong boundaries in recursion
- ✗ Forgetting to increase preorder index

30-Second Recap

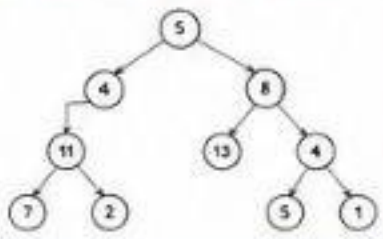
Root = first in preorder.
Find root in inorder.
Left = left part, Right = right part.
Recurse with new boundaries.

3. Path Sum (LC 112)

Check if there exists a root-to-leaf path with given sum.

IDEA:

Subtract node.val from sum while going down the tree.
If leaf node and sum == 0 → true.



Target Sum = 22
Path:
5 → 4 → 11 → 2
Sum = 22

Java Template:

```
boolean hasPathSum(TreeNode root, int targetSum) {
  if (root == null) return false;
  if (root.left == null && root.right == null)
    return targetSum == root.val;
  return hasPathSum(root.left, targetSum - root.val) ||
         hasPathSum(root.right, targetSum - root.val);
}
```

Top Leetcode Problems

- 112. Path Sum

Common Mistakes

- ✗ Not checking leaf condition properly
- ✗ Forgetting to subtract node value
- ✗ Returning true early without checking both paths

30-Second Recap

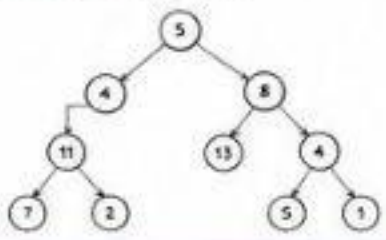
Go down, subtract value.
At leaf, check if remaining sum == 0.
Else try left or right.

4. Path Sum II (LC 113)

Find all root-to-leaf paths where sum == target.

IDEA:

Keep a list of current path.
When leaf and sum == 0 → add path to result.



Target = 22
Paths:
[5, 4, 11, 2]
[5, 8, 4, 5]

Java Template:

```
List<List<Integer>> pathSum(TreeNode root, int targetSum) {
  List<List<Integer>> res = new ArrayList<>();
  helper(root, targetSum, new ArrayList<>(), res);
  return res;
}
private void helper(TreeNode node, int sum,
        List<Integer> path, List<List<Integer>> res) {
  if (node == null) return;
  path.add(node.val);
  if (node.left == null && node.right == null && sum == node.val)
    res.add(new ArrayList<>(path));
  else {
    helper(node.left, sum - node.val, path, res);
    helper(node.right, sum - node.val, path, res);
  }
  path.remove(path.size() - 1); // backtrack
}
```

Top Leetcode Problems

- 113. Path Sum II

Common Mistakes

- ✗ Not backtracking (removing from path)
- ✗ Reusing same list reference
- ✗ Missing leaf condition

30-Second Recap

Add node to path.
At leaf & sum==val → add path.
Recurse left & right.
Remove last node (backtrack).

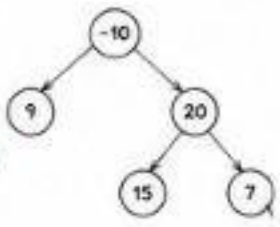
5. Maximum Path Sum (LC 124)

Find the maximum path sum in a binary tree.
Path can start and end at any nodes.

IDEA:

At each node:

- Take max left & right (if positive)
- Update global max with left + node.val + right
- Return node.val + max(left, right) to parent.



Max Path = 15 + 20 + 7 = 42

Java Template:

```
int maxPathSum(TreeNode root) {
  int[] maxSum = new int[]{Integer.MIN_VALUE};
  maxGain(root, maxSum);
  return maxSum[0];
}
private int maxGain(TreeNode node, int[] maxSum) {
  if (node == null) return 0;
  int left = Math.max(0, maxGain(node.left, maxSum));
  int right = Math.max(0, maxGain(node.right, maxSum));
  maxSum[0] = Math.max(maxSum[0], left + right + node.val);
  return node.val + Math.max(left, right);
}
```

Top Leetcode Problems

- 124. Binary Tree Maximum Path Sum

Common Mistakes

- ✗ Not using max(0, value) (taking negative paths)
- ✗ Updating global max in wrong place
- ✗ Returning left + right to parent

30-Second Recap

Compute left & right gains (ignore negatives).
Update global max with left + right + node.
Return node + max(left, right) to parent.

Summary of Advanced Tree Patterns

- ✓ LCA → Split problem in left & right.
- ✓ Build Tree → Use traversal properties.
- ✓ Path Sum → Modify data while going down.
- ✓ Path Sum II → Backtracking with path list.
- ✓ Max Path Sum → Take best from both sides.

AHA Moment

Break the big problem into smaller decisions at every node!